Refactoring: A Complete Guide

Concepts, Code Smells, Technical Debt, and Techniques

Source: refactoring.guru

What Is Refactoring?

Refactoring is a systematic process of improving code without creating new functionality that can transform a mess into clean code and simple design. It is the controllable process of systematically improving code without writing new functionality, and its main goal is to pay off technical debt. The mantra of refactoring is clean code and simple design.

Dirty Code vs Clean Code

Dirty code is the result of inexperience multiplied by tight deadlines, mismanagement, and nasty shortcuts taken during the development process. Clean code, by contrast, is code that is easy to read, understand, and maintain; it makes software development predictable and increases the quality of the resulting product.

Clean code has several defining features:

- Clean code is obvious for other programmers — poor variable naming, bloated classes and methods, and magic numbers all make code sloppy and difficult to grasp.
- Clean code doesn't contain duplication — each time you change duplicate code, you must remember to change every copy, which increases cognitive load and slows progress.
- Clean code contains a minimal number of classes and other moving parts — less code means less to keep in your head, less maintenance, and fewer bugs.
- Clean code passes all tests — you know your code is dirty when even a small percentage of tests fail, and you're in serious trouble if test coverage is zero.
- Clean code is easier and cheaper to maintain.

The Refactoring Process

Performing refactoring step-by-step and running tests after each change are key elements of refactoring that make it predictable and safe. This incremental discipline is what separates refactoring from simply rewriting code from scratch.

What Is a Code Smell?

A code smell is a surface indication that usually corresponds to a deeper problem in the system. Code smells are not bugs — they do not stop the program from functioning — but they are indicators of problems that can be addressed during refactoring. Code smells are easy to spot and fix, but they may just be symptoms of a deeper underlying problem with the code.

Refactoring.Guru groups code smells into five broad categories, each representing a different type of design weakness:

- **Bloaters:** Code, methods, and classes that have grown so large they are hard to work with. These smells accumulate gradually as a program evolves.
- **Object-Orientation Abusers:** Cases of incomplete or incorrect application of object-oriented programming principles.
- **Change Preventers:** Smells that mean a single change requires making many other changes elsewhere, making development more complicated and expensive.
- **Dispensables:** Something pointless and unneeded whose removal would make the code cleaner, more efficient, and easier to understand.
- **Couplers:** Smells that contribute to excessive coupling between classes, or that appear when coupling is replaced by excessive delegation.

Technical Debt

Everyone does their best to write excellent code from scratch, and there probably isn't a programmer who intentionally writes unclean code to the detriment of a project. But at what point does clean code become unclean? The metaphor of “technical debt” was originally suggested by Ward Cunningham to describe this drift.

If you take a loan from a bank, it lets you make purchases faster, but you pay extra for that speed in the form of interest — you don't just repay the principal, you also pay the additional interest on the loan. You can even rack up so much interest that it exceeds your total income, making full repayment impossible. The same thing can happen with code: you can temporarily speed up by skipping tests for a new feature, but this gradually slows your progress every day until you eventually pay off the debt by writing those tests.

Causes of Technical Debt

- **Business pressure:** Business circumstances might force you to roll out features before they're completely finished, leading to patches and kludges that hide unfinished parts of the project.
- **Lack of understanding of the consequences:** An employer might not understand that technical debt carries "interest" that slows down development as it accumulates, making it hard to justify time for refactoring.
- **Failing to combat strict coherence of components:** A project resembling a monolith rather than a set of individual modules, where changes to one part affect others, making team development harder.
- **Lack of tests:** Without immediate feedback, developers take quick but risky workarounds that may be deployed straight to production without prior testing, sometimes with catastrophic consequences.
- **Lack of documentation:** This slows down onboarding of new people and can grind development to a halt if key people leave the project.
- **Lack of interaction between team members:** If knowledge isn't distributed across the company, people end up working with outdated understanding, especially when junior developers are poorly mentored.
- **Long-term simultaneous development in several branches:** This leads to accumulating technical debt that increases further when changes are eventually merged.
- **Delayed refactoring:** As requirements change, parts of the code become obsolete and cumbersome; new code keeps being written against these obsolete parts, so delaying refactoring increases the amount of dependent code that must eventually be reworked.

- **Lack of compliance monitoring:** This happens when everyone on a project writes code however they see fit, without consistent standards.
- **Incompetence:** Sometimes a developer simply doesn't know how to write decent code.

How to Refactor

Refactoring is meant to improve the design, structure, and implementation details of software, while preserving its functionality. To refactor safely and effectively, follow this checklist:

- Refactor in small, controlled steps rather than making sweeping changes all at once.
- The code must become simpler and cleaner after every single refactoring step.
- Never mix refactoring with fixing bugs or adding new functionality — keep them as separate activities.
- Make sure all existing tests still pass after each refactoring step; if tests don't exist yet, write them first before refactoring.
- Commit or save working code frequently so you can revert quickly if a refactoring step introduces a problem.

This step-by-step approach keeps refactoring safe and predictable: since each change is small and verified immediately, if a mistake is made it is easy to find and undo without jeopardizing the rest of the codebase.

When to Refactor

- **Rule of Three:** The first time you do something, you just do it. The second time you do something similar, you wince at the duplication but do it anyway. The third time you do something similar, you refactor.
- **When adding a feature:** Refactoring helps you understand other people's code, and makes it easier to add new features since a clean, well-organized codebase is far simpler to extend.
- **When fixing a bug:** Bugs in code often live in the darkest, most tangled corners of a program; cleaning up the code helps you find bugs that were previously hidden by confusing structure.
- **During code review:** Code reviews may be the last chance to tidy up code before it's released to the rest of the team or into production, making refactoring a natural part of the review process.

Benefits of Refactoring

Refactoring is primarily done to fight technical debt. It transforms a mess into clean code and simple design, delivering several concrete advantages to a development team:

- **Helps you understand other people's code:** Refactoring makes code more uniform and predictable, which reduces the effort needed to understand unfamiliar code written by other team members.
- **Helps you find and fix bugs:** Since it helps you understand code better, refactoring also helps you spot bugs; even if not, it certainly helps you find them later.
- **Speeds up development in the long run:** In the short term, refactoring slows down feature delivery since it takes time, but a clean codebase pays dividends by making future feature development much faster.
- **Improves code review outcomes:** A code review is often the last opportunity to check code before it is released, and refactoring during review helps produce a cleaner final product.
- **Pays off technical debt:** Just as with a financial loan, technical debt accrues 'interest' that slows progress; regular refactoring keeps this interest from compounding and getting out of control.

In short, refactoring keeps code clean and simple, making software easier to understand, cheaper to maintain, and more resilient to bugs over its entire lifetime.

Composing Methods

Long methods hide their execution logic and become difficult to understand or modify. This group of techniques shortens methods, eliminates duplicate code, and creates a foundation for further improvements.

Extract Method

Problem: A fragment of code can be grouped together as a logical unit.

Solution: Move the fragment into a new method (or function) and replace the original code with a call to that method.

Why Refactor: Shorter methods with descriptive names are easier to read and understand. Isolating a fragment also makes it reusable elsewhere and reduces the risk of errors when modifying the logic.

Original Code	Refactored Code

`def print_owing(orders):`
`  print("Owing:")`
`  total = 0`
`  for o in orders:`
`  total += o.amount`
`  print(f"Total: {total}")`
	`def calculate_total(orders):`
	`  return sum(o.amount for o in orders)`
	
	`def print_owing(orders):`
	`  print("Owing:")`
	`  print(f"Total: {calculate_total(orders)}")`

Inline Method

Problem: A method's body is just as clear as its name, so the extra indirection is unnecessary.

Solution: Replace all calls to the method with its body, then delete the method.

Why Refactor: Removing unnecessary indirection makes code easier to follow when the extra method adds no clarity of its own.

Original Code	Refactored Code

`def more_than_five_deliveries(driver):`
`  return driver.deliveries > 5`
 	
`def get_rating(driver):`
`  return more_than_five_deliveries(driver)`
	`def get_rating(driver):`
	`  return driver.deliveries > 5`

Extract Variable

Problem: An expression is difficult to read or understand at a glance.

Solution: Assign the expression, or parts of it, to well-named variables that explain their purpose.

Why Refactor: Naming sub-expressions clarifies intent and makes complex conditions or calculations self-documenting.

Original Code	Refactored Code

`if platform.upper().startswith("MAC")`
`and browser.upper().startswith("IE"):`
`  do_something()`
	`is_mac =`
	`platform.upper().startswith("MAC")`
	`is_ie = browser.upper().startswith("IE")`
	`if is_mac and is_ie:`
	`  do_something()`

Inline Temp

Problem: A temporary variable is assigned once from a simple expression and used only that one time.

Solution: Replace all references to the variable with the expression itself and remove the variable.

Why Refactor: Removing an unnecessary variable simplifies the method and reduces clutter when the variable adds no explanatory value.

Original Code	Refactored Code

`base_price = order.quantity *`
`order.item_price`
`if base_price > 1000:`
`  return True`
	`if order.quantity * order.item_price >`
	`1000:`
	`  return True`

Replace Temp with Query

Problem: The result of an expression is stored in a local variable for later reuse in the method.

Solution: Move the whole expression into its own method and return the result, then call the method wherever the variable was used.

Why Refactor: Turning a temp into a method makes the calculation reusable by other methods and removes the risk of stale values.

Original Code	Refactored Code

`def price(self):`
`  base_price = self.quantity *`
`self.item_price`
`  if base_price > 1000:`
	`def base_price(self):`
	`  return self.quantity * self.item_price`
	
	`def price(self):`

`    return base_price * 0.95`
`  return base_price * 0.98`
	`  if self.base_price() > 1000:`
	`    return self.base_price() * 0.95`
	`  return self.base_price() * 0.98`

Split Temporary Variable

Problem: One local variable is reused to hold several different intermediate values within a method.

Solution: Use a separate variable for each distinct value so every variable has a single responsibility.

Why Refactor: Giving each purpose its own variable prevents confusion caused by reusing one variable for multiple, unrelated values.

Original Code	Refactored Code

`temp = 2 * (height + width)`
`print(temp)`
`temp = height * width`
`print(temp)`
	`perimeter = 2 * (height + width)`
	`print(perimeter)`
	`area = height * width`
	`print(area)`

Remove Assignments to Parameters

Problem: A value gets assigned to a parameter inside the body of the method.

Solution: Use a local variable instead of reassigning the parameter itself.

Why Refactor: Keeping parameters unchanged clarifies what a method's inputs represent throughout its body.

Original Code	Refactored Code

`def discount(input_val, quantity):`
`  if quantity > 50:`
`    input_val -= 2`
`  return input_val`
	`def discount(input_val, quantity):`
	`  result = input_val`
	`  if quantity > 50:`
	`    result -= 2`
	`  return result`

Replace Method with Method Object

Problem: A method is so long, and its local variables so interwoven, that Extract Method cannot be applied cleanly.

Solution: Turn the method into its own class so that local variables become fields, allowing the method to be split into several smaller ones.

Why Refactor: Turning fields into class attributes makes it possible to split an otherwise unmanageable long method into smaller pieces.

Original Code	Refactored Code

`def compute_price(order):`
`  base = order.quantity * order.item_price`
`  discount = base * order.discount_rate`
`  tax = (base - discount) * order.tax_rate`
`  return base - discount + tax`
	`class PriceCalculator:`
	`  def __init__(self, order):`
	`  self.order = order`
	
	`  def compute(self):`
	`  base = self._base()`
	`  discount = self._discount(base)`
	`  tax = self._tax(base, discount)`
	`  return base - discount + tax`
	
	`  def _base(self):`
	`  return self.order.quantity * self.order.item_price`
	
	`  def _discount(self, base):`
	`  return base * self.order.discount_rate`
	
	`  def _tax(self, base, discount):`
	`  return (base - discount) * self.order.tax_rate`

Substitute Algorithm

Problem: An existing algorithm needs to be replaced with a clearer or more efficient one.

Solution: Replace the body of the method with the new algorithm.

Why Refactor: Replacing a convoluted algorithm with a clearer or faster one improves both readability and performance.

Original Code	Refactored Code

`def find_person(people):`
`  for p in people:`
`  if p in ("Don", "John", "Kent"):`
`  return p`
`  return ""`
	`def find_person(people):`
	`  candidates = {"Don", "John", "Kent"}`
	`  return next((p for p in people if p in candidates), "")`

Moving Features between Objects

Functionality is sometimes distributed among classes in a less-than-ideal way. These techniques safely move behavior between classes, create new classes, and hide implementation details from external access.

Move Method

Problem: A method is used more by another class than by the one it is defined in.

Solution: Create a new method in the class that uses it most, and either delegate to it or remove the original.

Why Refactor: Placing a method where its data lives reduces coupling and makes the class relationships more logical.

Original Code	Refactored Code

`class Order:`
`  def __init__(self, items):`
`  self.items = items`
 	
`  def apply_discount(self):`
`  return sum(i.price for i in self.items) * 0.9`
	`class Order:`
	`  def __init__(self, items):`
	`  self.items = items`
	
	`class Discount:`
	`  @staticmethod`
	`  def apply(order):`
	`  return sum(i.price for i in order.items) * 0.9`

Move Field

Problem: A field is used more by another class than the one it currently belongs to.

Solution: Create a field in the class that uses it most and redirect all usages there.

Why Refactor: Relocating a field to its most relevant class improves cohesion and reduces unnecessary coupling.

Original Code	Refactored Code

`class Customer:`
`  def __init__(self, discount_rate):`
`  self.discount_rate = discount_rate`
 	
`class Account:`
`  def __init__(self, customer):`
`  self.customer = customer`
	`class Customer:`
	`  pass`
	
	`class Account:`
	`  def __init__(self, discount_rate):`
	`  self.discount_rate = discount_rate`

Extract Class

Problem: One class is doing the work that should belong to two classes.

Solution: Create a new class and move the relevant fields and methods from the old class into it.

Why Refactor: Splitting responsibilities into separate classes makes each class easier to understand, test, and reuse.

Original Code	Refactored Code

`class Person:`
`  def __init__(self, name, area_code, number):`
`  self.name = name`
`  self.area_code = area_code`
`  self.number = number`
	`class PhoneNumber:`
	`  def __init__(self, area_code, number):`
	`  self.area_code = area_code`
	`  self.number = number`
	
	`class Person:`
	`  def __init__(self, name, area_code, number):`
	`  self.name = name`
	`  self.phone = PhoneNumber(area_code, number)`

Inline Class

Problem: A class does very little and is not worth keeping on its own.

Solution: Move all of its features into another class and delete the near-empty class.

Why Refactor: Removing an unnecessary class reduces complexity when a class no longer justifies its own existence.

Original Code	Refactored Code

`class PhoneNumber:`
`  def __init__(self, number):`
`  self.number = number`
 	
`class Person:`
`  def __init__(self, name, number):`
`  self.name = name`
`  self.phone = PhoneNumber(number)`
	`class Person:`
	`  def __init__(self, name, number):`
	`  self.name = name`
	`  self.number = number`

Hide Delegate

Problem: A client calls a delegate class through a field of the server class, coupling the client to the delegate.

Solution: Create a method on the server class that hides the delegate from the client.

Why Refactor: Hiding the delegate reduces coupling since clients no longer need to know about intermediate objects.

Original Code	Refactored Code

`class Department:`
`  def __init__(self, manager):`
`  self.manager = manager`
 	
`class Person:`
`  def __init__(self, department):`
`  self.department = department`
 	
`# Client code`
`manager = person.department.manager`
	`class Person:`
	`  def __init__(self, department):`
	`  self.department = department`
	
	`  def manager(self):`
	`  return self.department.manager`
	
	`# Client code`
	`manager = person.manager()`

Remove Middle Man

Problem: A class has too many simple delegating methods that just forward calls to another class.

Solution: Let the client call the end method directly instead of going through the delegating class.

Why Refactor: Removing excessive indirection simplifies the code when delegation provides no real benefit.

Original Code	Refactored Code

`class Person:`
`  def manager(self):`
`  return self.department.manager`
 	
`manager = person.manager()`
	`manager = person.department.manager`

Introduce Foreign Method

Problem: A utility class you cannot modify is missing a method that you need.

Solution: Add the needed method to a client class instead, passing an instance of the utility class as an argument.

Why Refactor: This lets you add missing functionality without modifying code you don't own or can't change.

Original Code	Refactored Code

`from datetime import datetime`
`tomorrow = datetime.now()  # no built-in "next day" helper available directly`
	`from datetime import datetime, timedelta`
	
	`def next_day(date):`
	`  return date + timedelta(days=1)`
	
	`tomorrow = next_day(datetime.now())`

Introduce Local Extension

Problem: A utility class needs several additional methods, but you cannot modify its source.

Solution: Create a new class that extends or wraps the utility class and add the extra methods there.

Why Refactor: This provides a clean way to add multiple new methods to a class you cannot modify directly.

Original Code	Refactored Code

`from datetime import datetime`
`d = datetime.now()  # no next_business_day() method exists`
	`from datetime import datetime, timedelta`
	
	`class ExtendedDate(datetime):`
	`  def next_business_day(self):`
	`  result = self + timedelta(days=1)`
	`  while result.weekday() >= 5:`
	`  result += timedelta(days=1)`
	`  return result`

Organizing Data

This group assists with data handling by replacing primitive values with rich, class-based functionality, and by untangling class associations to make classes more reusable and portable.

Self Encapsulate Field

Problem: Direct access to a field limits flexibility, such as lazy initialization or validation.

Solution: Create a getter and setter for the field and use them even within the class itself.

Why Refactor: Using accessor methods, even internally, allows you to add validation or lazy loading later without breaking callers.

Original Code	Refactored Code

`class Circle:`
`  def __init__(self, radius):`
`  self.radius = radius`
 	
`  def area(self):`
`  return 3.14159 * self.radius ** 2`
	`class Circle:`
	`  def __init__(self, radius):`
	`  self._radius = radius`
	
	`  def get_radius(self):`
	`  return self._radius`
	
	`  def area(self):`
	`  return 3.14159 * self.get_radius() ** 2`

Replace Data Value with Object

Problem: A simple data field needs additional behavior or data associated with it.

Solution: Turn the field into its own object.

Why Refactor: Wrapping a primitive in a class allows you to add validation and behavior specific to that value.

Original Code	Refactored Code

`class Order:`
`  def __init__(self, customer_name):`
`  self.customer_name = customer_name`
	`class Customer:`
	`  def __init__(self, name):`
	`  self.name = name`
	
	`class Order:`
	`  def __init__(self, customer_name):`
	`  self.customer = Customer(customer_name)`

Change Value to Reference

Problem: Many identical instances of one class need to be replaced with a single shared object.

Solution: Convert the value object into a reference object managed by a factory or registry.

Why Refactor: Sharing a single instance avoids inconsistent data across multiple copies of what should be the same entity.

Original Code	Refactored Code

`class Customer:`
`  def __init__(self, name):`
`  self.name = name`
 	
`c1 = Customer("John")`
`c2 = Customer("John")  # duplicate instance`
	`customers = {}`
	
	`def get_customer(name):`
	`  if name not in customers:`
	`  customers[name] = Customer(name)`
	`  return customers[name]`
	
	`c1 = get_customer("John")`
	`c2 = get_customer("John")  # same instance`

Change Reference to Value

Problem: A small, immutable reference object is awkward to manage and rarely changes.

Solution: Turn it into a value object that can be freely copied.

Why Refactor: Value objects are simpler to manage, can be freely copied, and avoid issues from shared mutable state.

Original Code	Refactored Code

`class Money:`
`  def __init__(self, amount, currency):`
`  self.amount = amount`
`  self.currency = currency`
	`from dataclasses import dataclass`
	
	`@dataclass(frozen=True)`
	`class Money:`
	`  amount: float`
	`  currency: str`

Replace Array with Object

Problem: An array holds elements that each mean something different.

Solution: Replace the array with an object that has a named field for each element.

Why Refactor: Named fields make code more self-explanatory than obscure numeric indices into an array.

Original Code	Refactored Code

row = ["John", 28]
name = row[0]
age = row[1]
	class Employee:
	  def __init__(self, name, age):
	  self.name = name
	  self.age = age
	
	emp = Employee("John", 28)

Duplicate Observed Data

Problem: Domain data is stored in GUI classes, tying business logic to the interface.

Solution: Move the data into separate domain classes and keep them synchronized with the GUI.

Why Refactor: Separating domain logic from the GUI makes business rules testable independent of the interface.

Original Code	Refactored Code

class SalesView:
  def __init__(self, amount):
  self.amount = amount
 	
  def render(self):
  print(f"Sales: {self.amount}")
	class SalesModel:
	  def __init__(self, amount):
	  self.amount = amount
	
	class SalesView:
	  def render(self, model:
	SalesModel):
	  print(f"Sales: {model.amount}")

Change Unidirectional Association to Bidirectional

Problem: Two classes need to use each other's features, but only one holds a reference to the other.

Solution: Add a back-reference field to the second class.

Why Refactor: This enables both classes to access each other conveniently when both directions are genuinely needed.

Original Code	Refactored Code

class Customer:
  pass
 	
class Order:
  def __init__(self, customer):
  self.customer = customer
	class Customer:
	  def __init__(self):
	  self.orders = []
	
	class Order:
	  def __init__(self, customer):

	self.customer = customer customer.orders.append(self)

Change Bidirectional Association to Unidirectional

Problem: A two-way class association exists, but one direction of the link is never actually used.

Solution: Remove the unused reference to simplify the relationship.

Why Refactor: Removing an unused reference reduces coupling and simplifies the object graph.

Original Code	Refactored Code

class Customer:
  def __init__(self):
  self.orders = [] # never actually queried
 	
class Order:
  def __init__(self, customer):
  self.customer = customer
  customer.orders.append(self)
	class Customer:
	  pass
	
	class Order:
	  def __init__(self, customer):
	  self.customer = customer

Replace Magic Number with Symbolic Constant

Problem: A literal number appears in code without explanation of its meaning.

Solution: Replace it with a well-named constant.

Why Refactor: Named constants make the meaning and intent of literal values clear at a glance.

Original Code	Refactored Code
area = radius * radius * 3.14159	PI = 3.14159 area = radius * radius * PI

Encapsulate Field

Problem: A public field can be modified from outside the class without any checks.

Solution: Make the field private and provide access methods.

Why Refactor: Private fields with access methods let you validate and control how data is modified.

Original Code	Refactored Code

class Account:
  def __init__(self, balance):
  self.balance = balance
 	
acc = Account(100)
acc.balance = -50 # *invalid state possible*
	class Account:
	  def __init__(self, balance):
	  self._balance = balance
	
	  @property
	  def balance(self):
	  return self._balance
	
	  @balance.setter
	  def balance(self, value):
	  if value < 0:
	  raise ValueError("Balance cannot be negative")
	  self._balance = value

Encapsulate Collection

Problem: A method returns a raw collection field, allowing external code to modify it directly.

Solution: Return a read-only view and provide dedicated add/remove methods.

Why Refactor: Preventing direct mutation of a collection field protects the object's internal state from invalid changes.

Original Code	Refactored Code

class Team:
  def __init__(self):
  self.players = []
 	
team = Team()
team.players.append("Sam") # *uncontrolled mutation*
	class Team:
	  def __init__(self):
	  self._players = []
	
	  def players(self):
	  return tuple(self._players)
	
	  def add_player(self, player):
	  self._players.append(player)

Replace Type Code with Class

Problem: A field holds a raw code (int or string) that represents a type, with no real type-safety.

Solution: Introduce a dedicated class or constants to represent the type explicitly.

Why Refactor: A dedicated class prevents invalid values and gives the type code its own identity and behavior.

Original Code	Refactored Code

`class Car:`
`  def __init__(self, color):`
`  self.color = color  # e.g. "RED"`
	`class Color:`
	`  RED, GREEN, BLUE = "RED", "GREEN", "BLUE"`
	
	`class Car:`
	`  def __init__(self, color=Color.RED):`
	`  self.color = color`

Replace Type Code with Subclasses

Problem: A type code changes an object's behavior via conditional logic scattered around the code.

Solution: Create a subclass for each value of the type code and use polymorphism instead.

Why Refactor: Polymorphism removes repetitive conditional logic and makes it easy to add new types safely.

Original Code	Refactored Code

`class Employee:`
`  def __init__(self, emp_type):`
`  self.emp_type = emp_type`
 	
`  def pay(self):`
`  if self.emp_type == "ENGINEER":`
`  return 5000`
`  elif self.emp_type == "MANAGER":`
`  return 8000`
	`class Employee:`
	`  def pay(self):`
	`  raise NotImplementedError`
	
	`class Engineer(Employee):`
	`  def pay(self):`
	`  return 5000`
	
	`class Manager(Employee):`
	`  def pay(self):`
	`  return 8000`

Replace Type Code with State/Strategy

Problem: A type code determines behavior, but the class hierarchy cannot be changed, or the type can change at runtime.

Solution: Introduce a state or strategy object that can be swapped at runtime to change behavior.

Why Refactor: This allows behavior to change dynamically at runtime, unlike fixed inheritance-based solutions.

Original Code	Refactored Code

class Order:
  def __init__(self, status):
  self.status = status # "PENDING", "SHIPPED", etc.
 	
  def handle(self):
  if self.status == "PENDING":
  print("Processing order")
	class OrderState:
	  def handle(self, order):
	  raise NotImplementedError
	
	class PendingState(OrderState):
	  def handle(self, order):
	  print("Processing order")
	
	class Order:
	  def __init__(self):
	  self.state = PendingState()
	
	  def handle(self):
	  self.state.handle(self)

Replace Subclass with Fields

Problem: Subclasses differ only by constant-returning methods and add no real behavior.

Solution: Replace the subclasses with fields on the parent class.

Why Refactor: Removing needless subclasses that only differ by constants simplifies the class hierarchy.

Original Code	Refactored Code

class Person:
  pass
 	
class Male(Person):
  def is_male(self):
  return True
 	
class Female(Person):
  def is_male(self):
  return False
	class Person:
	  def __init__(self, is_male):
	  self.is_male = is_male

Simplifying Conditional Expressions

Conditional logic tends to grow more tangled over time. These techniques simplify complex branching so intent remains clear as code evolves.

Decompose Conditional

Problem: A complex conditional (if-then-else) obscures what it actually checks and does.

Solution: Extract the condition and each branch into separate, clearly named methods.

Why Refactor: Breaking down a complex conditional makes the purpose of each branch immediately clear.

Original Code	Refactored Code

`if date.month in (12, 1, 2):`
`  charge = quantity * winter_rate + winter_service_charge`
`else:`
`  charge = quantity * summer_rate`
	`def is_winter(date):`
	`  return date.month in (12, 1, 2)`
	
	`def winter_charge(quantity):`
	`  return quantity * winter_rate + winter_service_charge`
	
	`def summer_charge(quantity):`
	`  return quantity * summer_rate`
	
	`charge = winter_charge(quantity) if is_winter(date) else summer_charge(quantity)`

Consolidate Conditional Expression

Problem: A series of conditional checks all lead to the same result or action.

Solution: Combine the conditions into a single expression, typically extracted into its own method.

Why Refactor: A single, well-named check communicates intent better than several scattered conditionals with the same outcome.

Original Code	Refactored Code

`if seniority < 2:`
`  return 0`
`if months_disabled > 12:`
`  return 0`
`if is_part_time:`
`  return 0`
	`def is_not_eligible_for_disability(seniority, months_disabled, is_part_time):`
	`  return seniority < 2 or months_disabled > 12 or is_part_time`
	
	`if is_not_eligible_for_disability(seniority, months_disabled, is_part_time):`
	`  return 0`

Consolidate Duplicate Conditional Fragments

Problem: Identical code appears inside every branch of a conditional.

Solution: Move the duplicate code outside of the conditional entirely.

Why Refactor: Removing duplicate code from inside branches keeps logic DRY (Don't Repeat Yourself) and easier to change.

Original Code	Refactored Code

`if is_special:`
`  total = price * 0.95`
`  send_email()`
`else:`
`  total = price`
`  send_email()`
	`total = price * 0.95 if is_special else price`
	`send_email()`

Remove Control Flag

Problem: A boolean variable is used to control when a loop should exit, adding unnecessary complexity.

Solution: Replace the control flag with break, continue, or return statements.

Why Refactor: Direct control flow using break/continue/return is easier to trace than a flag variable.

Original Code	Refactored Code

`found = False`
`for item in items:`
`  if not found:`
`  if item == target:`
`  found = True`
`  result = item`
	`def find_item(items, target):`
	`  for item in items:`
	`  if item == target:`
	`  return item`
	`  return None`

Replace Nested Conditional with Guard Clauses

Problem: Deeply nested if-else blocks make it hard to see the normal path of execution.

Solution: Use early return 'guard clauses' to handle special cases first, keeping the main logic flat.

Why Refactor: Guard clauses make the main logic path more visible by handling exceptional cases upfront.

Original Code	Refactored Code

def get_payout(employee):
  result = 0
  if employee.is_separated:
  if employee.is_retired:
  result = 0
  else:
  result = separation_pay(employee)
  else:
  result = normal_pay(employee)
  return result
	def get_payout(employee):
	  if employee.is_separated and employee.is_retired:
	  return 0
	  if employee.is_separated:
	  return separation_pay(employee)
	  return normal_pay(employee)

Replace Conditional with Polymorphism

Problem: Behavior is selected through a conditional based on an object's type or state.

Solution: Move each conditional branch into an overriding method in a corresponding subclass.

Why Refactor: Polymorphism eliminates repetitive type-checking logic and makes adding new behaviors safer and easier.

Original Code	Refactored Code

class Bird:
  def __init__(self, kind):
  self.kind = kind
 	
  def speed(self):
  if self.kind == "European":
  return 35
  elif self.kind == "African":
  return 40
	class Bird:
	  def speed(self):
	  raise NotImplementedError
	
	class EuropeanSwallow(Bird):
	  def speed(self):
	  return 35
	
	class AfricanSwallow(Bird):
	  def speed(self):
	  return 40

Introduce Null Object

Problem: Repeated checks for None scatter throughout the code wherever an object might be missing.

Solution: Provide a Null Object that implements default, do-nothing behavior instead of returning None.

Why Refactor: A Null Object removes repetitive None checks scattered throughout the code, simplifying client logic.

Original Code	Refactored Code

def get_discount(customer_id):
  customer = customers.get(customer_id)
  if customer is None:
  return 0
  return customer.get_discount()
	class NullCustomer:
	  name = "Unknown"
	  def get_discount(self):
	  return 0
	
	def find_customer(customer_id):
	  return customers.get(customer_id, NullCustomer())
	
	discount = find_customer(customer_id).get_discount()

Introduce Assertion

Problem: A section of code relies on an assumption about state that is never explicitly stated.

Solution: Make the assumption explicit with an assertion.

Why Refactor: Explicit assertions document assumptions and catch violations early, before they cause subtle bugs.

Original Code	Refactored Code

def calculate_discount(price):
  return price * 0.9 *# assumes price is non-negative*
	def calculate_discount(price):
	  assert price >= 0, "Price cannot be negative"
	  return price * 0.9

Simplifying Method Calls

These techniques simplify method interfaces and make interactions between classes easier to understand and change.

Add Parameter

Problem: A method needs additional information from its caller to do its job.

Solution: Add a new parameter to pass that information in.

Why Refactor: Adding a parameter provides the method with data it genuinely needs to do its job correctly.

Original Code	Refactored Code
`def send_email(to, subject):` `  pass`	`def send_email(to, subject, body):` `  pass`

Remove Parameter

Problem: A method has a parameter that is no longer used in its body.

Solution: Remove the unused parameter.

Why Refactor: Removing unused parameters keeps method signatures honest and easier to understand.

Original Code	Refactored Code
`def greet(name, title):` `  print(f"Hello, {name}")`	`def greet(name):` `  print(f"Hello, {name}")`

Rename Method

Problem: A method's name does not clearly describe what it does.

Solution: Rename the method to reflect its true purpose.

Why Refactor: A descriptive name makes the intent of a method obvious without needing to read its implementation.

Original Code	Refactored Code
`def calc(items):` `  return sum(i.price for i in items)`	`def get_total(items):` `  return sum(i.price for i in items)`

Separate Query from Modifier

Problem: A single method both returns a value and changes the state of the object.

Solution: Create two separate methods: one to query the value, one to modify state.

Why Refactor: Separating queries from modifiers avoids surprising side effects when a caller only wants to read a value.

Original Code	Refactored Code

`def get_total_and_apply_discount(self):`
`  self._total *= 0.9`
`  return self._total`
	`def get_total(self):`
	`  return self._total`
	
	`def apply_discount(self):`

	self._total *= 0.9

Parameterize Method

Problem: Several methods perform similar actions that differ only by an internal numeric or string value.

Solution: Merge them into one method that accepts the differing value as a parameter.

Why Refactor: Merging similar methods into one reduces duplication and centralizes related logic in a single place.

Original Code	Refactored Code

def raise_5_percent(amount):
  return amount * 1.05
 	
def raise_10_percent(amount):
  return amount * 1.10
	def adjust_salary(amount, factor):
	  return amount * factor

Introduce Parameter Object

Problem: A group of parameters is consistently passed together across several methods.

Solution: Replace the group with a single object that carries all of them.

Why Refactor: Bundling related parameters into an object simplifies method signatures and clarifies their relationship.

Original Code	Refactored Code

def get_bookings(start_date, end_date, guest_name, guest_email):
  pass
	from dataclasses import dataclass
	
	@dataclass
	class BookingRequest:
	  start_date: str
	  end_date: str
	  guest_name: str
	  guest_email: str
	
	def get_bookings(request: BookingRequest):
	  pass

Preserve Whole Object

Problem: Several values are extracted from an object just to pass them individually to another method.

Solution: Pass the whole object instead of the extracted values.

Why Refactor: Passing the whole object instead of extracted fields keeps method signatures shorter and more flexible to future changes.

Original Code	Refactored Code

`def within_range(low, high, date):`
`  return low <= date <= high`
 	
`result = within_range(date_range.low, date_range.high, date)`
	`def within_range(date_range, date):`
	`  return date_range.low <= date <= date_range.high`
	
	`result = within_range(date_range, date)`

Remove Setting Method

Problem: A field should never change after the object is constructed, yet a setter still exists for it.

Solution: Remove the setter method for that field.

Why Refactor: Removing setters for immutable fields protects object invariants and prevents unintended state changes.

Original Code	Refactored Code

`class Employee:`
`  def __init__(self, employee_id):`
`  self.id = employee_id`
 	
`  def set_id(self, new_id):`
`  self.id = new_id`
	`class Employee:`
	`  def __init__(self, employee_id):`
	`  self._id = employee_id`

Replace Parameter with Explicit Methods

Problem: A method behaves differently depending on the value of a parameter that acts like a switch.

Solution: Create a separate, explicitly named method for each possible value.

Why Refactor: Explicit methods are easier to understand and use correctly than a generic method controlled by a flag-like parameter.

Original Code	Refactored Code

def set_direction(direction):
  if direction == "NORTH":
  pass
  elif direction == "SOUTH":
  pass
	def set_north():
	  pass
	
	def set_south():
	  pass

Replace Parameter with Method Call

Problem: A caller performs a computation just to pass the result as a parameter that the method could compute itself.

Solution: Have the method call the necessary query internally rather than accept it as a parameter.

Why Refactor: Letting the method compute the value itself avoids redundant work and errors from stale caller-provided values.

Original Code	Refactored Code

def get_price(base_price, discount_level):
  return base_price * discount_level
 	
price = get_price(self.base_price(), self.discount_level())
	def get_price(self):
	  return self.base_price() * self.discount_level()

Hide Method

Problem: A method is used only within its own class but is exposed as public.

Solution: Mark the method as private or protected.

Why Refactor: Restricting visibility protects internal implementation details from being relied upon externally.

Original Code	Refactored Code

class Report:
  def format_row(self, row):
  pass
	class Report:
	  def _format_row(self, row):
	  pass

Replace Constructor with Factory Method

Problem: Creating an object requires extra logic beyond simple field assignment.

Solution: Introduce a factory method that encapsulates the object creation logic.

Why Refactor: A factory method can encapsulate complex creation logic and choose the appropriate subclass or configuration.

Original Code	Refactored Code

class Employee:
  def __init__(self, name, emp_type):
  self.name = name
  self.type = emp_type
 	
e = Employee("John", "ENGINEER")
	class Employee:
	  def __init__(self, name):
	  self.name = name
	  self.type = None
	
	  @staticmethod
	  def create_engineer(name):
	  emp = Employee(name)
	  emp.type = "ENGINEER"
	  return emp
	
	e = Employee.create_engineer("John")

Replace Error Code with Exception

Problem: A method returns a special error code that the caller must remember to check.

Solution: Raise an exception instead of returning an error code.

Why Refactor: Exceptions can't be silently ignored the way return codes can, making error handling more robust.

Original Code	Refactored Code

def withdraw(balance, amount):
  if amount > balance:
  return -1
  return balance - amount
	def withdraw(balance, amount):
	  if amount > balance:
	  raise ValueError("Insufficient funds")
	  return balance - amount

Replace Exception with Test

Problem: An exception is used to handle a condition the caller could easily have checked beforehand.

Solution: Replace the exception handling with a simple conditional test.

Why Refactor: A simple check avoids the overhead and complexity of exception handling for conditions that are easy to predict beforehand.

Original Code	Refactored Code

def get_value(d, key, default=None):
  try:
  return d[key]
  except KeyError:
  return default
	def get_value(d, key, default=None):
	  if key in d:
	  return d[key]
	  return default

Dealing with Generalization

This category involves moving functionality along a class inheritance hierarchy, creating new classes and interfaces, and replacing inheritance with delegation or vice versa.

Pull Up Field

Problem: Two subclasses have identical fields that could be shared.

Solution: Move the field up to their common superclass.

Why Refactor: Consolidating a shared field in the superclass avoids duplication across sibling subclasses.

Original Code	Refactored Code

class Manager:
  def __init__(self, name):
  self.name = name
 	
class Engineer:
  def __init__(self, name):
  self.name = name
	class Employee:
	  def __init__(self, name):
	  self.name = name
	
	class Manager(Employee):
	  pass
	
	class Engineer(Employee):
	  pass

Pull Up Method

Problem: Subclasses contain methods that produce identical results.

Solution: Move the method up to the shared superclass.

Why Refactor: Sharing a common method in the superclass avoids maintaining multiple near-identical implementations.

Original Code	Refactored Code

class Manager:
  def get_name(self):
  return self.name
	class Employee:
	  def get_name(self):
	  return self.name

class Engineer:
  def get_name(self):
  return self.name
	class Manager(Employee):
	  pass
	
	class Engineer(Employee):
	  pass

Pull Up Constructor Body

Problem: Subclass constructors share largely identical code.

Solution: Move the common construction logic into the superclass constructor and call it via super().

Why Refactor: Centralizing shared construction logic avoids duplicated initialization code across subclasses.

Original Code	Refactored Code

class Manager:
  def __init__(self, name, level):
  self.name = name
  self.level = level
 	
class Engineer:
  def __init__(self, name, skill):
  self.name = name
  self.skill = skill
	class Employee:
	  def __init__(self, name):
	  self.name = name
	
	class Manager(Employee):
	  def __init__(self, name, level):
	  super().__init__(name)
	  self.level = level
	
	class Engineer(Employee):
	  def __init__(self, name, skill):
	  super().__init__(name)
	  self.skill = skill

Push Down Field

Problem: A field in the superclass is only relevant to one subclass.

Solution: Move the field down into the subclass that actually needs it.

Why Refactor: Moving a field down keeps the superclass focused on truly shared state, relevant to all subclasses.

Original Code	Refactored Code

class Employee:
  def __init__(self, name, budget_limit):
  self.name = name
  self.budget_limit = budget_limit #
	class Employee:
	  def __init__(self, name):
	  self.name = name
	
	class Manager(Employee):

only Managers need this	`def __init__(self, name, budget_limit):`
	`  super().__init__(name)`
	`  self.budget_limit = budget_limit`

Push Down Method

Problem: A method in the superclass is relevant to only one subclass.

Solution: Move the method down into the subclass that needs it.

Why Refactor: Moving a method down keeps the superclass interface clean and relevant to every subclass.

Original Code	Refactored Code

`class Employee:`
`  def approve_budget(self):`
`  pass  # only relevant to managers`
	`class Employee:`
	`  pass`
	
	`class Manager(Employee):`
	`  def approve_budget(self):`
	`  pass`

Extract Subclass

Problem: A class has features that are used only in certain instances.

Solution: Create a subclass and move those rarely used features into it.

Why Refactor: Isolating rarely-used features into a subclass keeps the base class simpler for the common case.

Original Code	Refactored Code

`class Employee:`
`  def __init__(self, name, hourly_rate=None):`
`  self.name = name`
`  self.hourly_rate = hourly_rate  # only used by consultants`
	`class Employee:`
	`  def __init__(self, name):`
	`  self.name = name`
	
	`class Consultant(Employee):`
	`  def __init__(self, name, hourly_rate):`
	`  super().__init__(name)`
	`  self.hourly_rate = hourly_rate`

Extract Superclass

Problem: Two classes share common fields and methods with no shared parent.

Solution: Create a superclass and move the shared elements into it.

Why Refactor: A shared superclass eliminates duplicate fields and methods across related classes.

Original Code	Refactored Code

`class Person:`
`  def __init__(self, name):`
`  self.name = name`
 	
`class Company:`
`  def __init__(self, name):`
`  self.name = name`
	`class Party:`
	`  def __init__(self, name):`
	`  self.name = name`
	
	`class Person(Party):`
	`  pass`
	
	`class Company(Party):`
	`  pass`

Extract Interface

Problem: Several clients use only a portion of a class's interface, or two classes share part of an interface.

Solution: Extract the shared portion into a separate interface or abstract base class.

Why Refactor: A shared interface lets clients depend on an abstraction instead of concrete implementations, improving flexibility.

Original Code	Refactored Code

`class Employee:`
`  def pay(self):`
`  return self.salary`
 	
`class Contractor:`
`  def pay(self):`
`  return self.invoice_amount`
	`from abc import ABC, abstractmethod`
	
	`class Payable(ABC):`
	`  @abstractmethod`
	`  def pay(self): ...`
	
	`class Employee(Payable):`
	`  def pay(self):`
	`  return self.salary`
	
	`class Contractor(Payable):`
	`  def pay(self):`
	`  return self.invoice_amount`

Collapse Hierarchy

Problem: A subclass and its superclass have become nearly identical over time.

Solution: Merge the two classes together into one.

Why Refactor: Merging near-identical classes reduces unnecessary complexity in the class hierarchy.

Original Code	Refactored Code

`class Employee:`
`  def __init__(self, name):`
`  self.name = name`
 	
`class TemporaryEmployee(Employee):`
`  pass  # no longer adds anything`
	`class Employee:`
	`  def __init__(self, name):`
	`  self.name = name`

Form Template Method

Problem: Two subclasses implement similar algorithms with the same overall structure but differing steps.

Solution: Move the common structure to a superclass template method and let subclasses override the varying steps.

Why Refactor: A template method captures a shared algorithm structure once, letting subclasses vary only the steps that differ.

Original Code	Refactored Code

`class CSVReport:`
`  def generate(self):`
`  data = ["row1", "row2"]`
`  return ",".join(data)`
 	
`class JSONReport:`
`  def generate(self):`
`  data = ["row1", "row2"]`
`  return str(data)`
	`class ReportGenerator:`
	`  def generate(self):`
	`  return self.format(self.fetch_data())`
	
	`  def fetch_data(self):`
	`  raise NotImplementedError`
	
	`  def format(self, data):`
	`  raise NotImplementedError`
	
	`class CSVReport(ReportGenerator):`
	`  def fetch_data(self):`
	`  return ["row1", "row2"]`
	
	`  def format(self, data):`
	`  return ",".join(data)`

Replace Inheritance with Delegation

Problem: A subclass uses only a small part of its superclass's interface, making inheritance misleading.

Solution: Replace inheritance with a field referencing an instance of the former superclass, and delegate calls to it.

Why Refactor: Delegation avoids inheriting unwanted behavior that doesn't logically belong to the subclass.

Original Code	Refactored Code

`class Vehicle:`
`  def start(self):`
`  print("Starting")`
`  def fly(self):`
`  print("Flying")  # Car should not inherit this`
 	
`class Car(Vehicle):`
`  pass`
	`class Vehicle:`
	`  def start(self):`
	`  print("Starting")`
	
	`class Car:`
	`  def __init__(self):`
	`  self._vehicle = Vehicle()`
	
	`  def start(self):`
	`  self._vehicle.start()`

Replace Delegation with Inheritance

Problem: A class delegates all of its work to another class, adding no value of its own.

Solution: Make the class inherit from the delegate instead of holding a reference to it.

Why Refactor: Inheritance simplifies the design when a class delegates all its behavior with no added value of its own.

Original Code	Refactored Code

`class Vehicle:`
`  def start(self):`
`  print("Starting")`
 	
`class Car:`
`  def __init__(self):`
`  self._vehicle = Vehicle()`
 	
`  def start(self):`
`  self._vehicle.start()`
	`class Vehicle:`
	`  def start(self):`
	`  print("Starting")`
	
	`class Car(Vehicle):`
	`  pass`

Code Smells

Indicators of Deeper Problems in Code, with Python Examples and Fixes

Source: refactoring.guru/refactoring/smells

A code smell is a surface indication that usually corresponds to a deeper problem in the system. Code smells are not bugs; they do not prevent the program from functioning, but they indicate weaknesses in design that may slow down development or increase the risk of bugs and failures. Refactoring.Guru groups code smells into five categories, each addressed below with a Python example of the smell and the corresponding refactored fix.

Smell Categories Covered

- Bloaters
- Object-Orientation Abusers
- Change Preventers
- Dispensables
- Couplers

Bloaters

Bloaters are code, methods and classes that have increased to such gargantuan proportions that they are hard to work with. Usually these smells do not crop up right away, rather they accumulate over time as the program evolves.

Long Method

Smell: A method contains too many lines of code. Generally, any method longer than ten lines should make you start asking questions.

Typical Fix: Fixed primarily with Extract Method.

Code with Smell	Refactored Code

def process_order(order):
  total = 0
  for item in order.items:
  total += item.price * item.qty
  if order.customer.is_member:
  total *= 0.9
  tax = total * 0.08
  total += tax
  print(f"Shipping to {order.customer.address}")
  print(f"Total: {total}")
  return total
	def calculate_subtotal(order):
	  return sum(i.price * i.qty for i in order.items)
	
	def apply_member_discount(total, customer):
	  return total * 0.9 if customer.is_member else total
	
	def add_tax(total):
	  return total + total * 0.08
	
	def process_order(order):
	  total = calculate_subtotal(order)
	  total = apply_member_discount(total, order.customer)
	  total = add_tax(total)
	  print(f"Shipping to {order.customer.address}")
	  print(f"Total: {total}")
	  return total

Large Class

Smell: A class contains many fields, methods, or lines of code, taking on too many responsibilities.

Typical Fix: Fixed primarily with Extract Class or Extract Subclass.

Code with Smell	Refactored Code

class Employee:
  def __init__(self, name, salary, address, phone):
	class ContactInfo:
	  def __init__(self, address, phone):
	  self.address = address

```
        self.name = name
        self.salary = salary
        self.address = address
        self.phone = phone

    def calculate_pay(self):
        return self.salary

    def print_address_label(self):
        print(self.address)

    def send_sms(self, message):
        print(f"SMS to {self.phone}:
{message}")
```

```
        self.phone = phone

    def print_address_label(self):
        print(self.address)

    def send_sms(self, message):
        print(f"SMS to {self.phone}:
{message}")

class Employee:
    def __init__(self, name, salary,
contact: ContactInfo):
        self.name = name
        self.salary = salary
        self.contact = contact

    def calculate_pay(self):
        return self.salary
```

Primitive Obsession

Smell: Use of primitives instead of small objects for simple tasks (like currency or phone numbers), use of constants for coded information, or use of string constants as field names.

Typical Fix: Fixed with Replace Data Value with Object, Replace Type Code with Class, or Extract Class.

Code with Smell

```
USER_ADMIN_ROLE = 1

def create_user(name, role_code):
    return {"name": name, "role":
role_code}

user = create_user("John",
USER_ADMIN_ROLE)
```

Refactored Code

```
class Role:
    ADMIN = "ADMIN"
    MEMBER = "MEMBER"

class User:
    def __init__(self, name, role: str):
        self.name = name
        self.role = role

user = User("John", Role.ADMIN)
```

Long Parameter List

Smell: A method has more than three or four parameters, making it hard to understand and call correctly.

Typical Fix: Fixed with Introduce Parameter Object, Preserve Whole Object, or Replace Parameter with Method Call.

Code with Smell	Refactored Code

def create_booking(guest_name, guest_email, check_in, check_out, room_type, num_guests):
  pass
 	
create_booking("John", "john@mail.com", "2026-08-01", "2026-08-05", "Deluxe", 2)
	from dataclasses import dataclass
	
	@dataclass
	class BookingRequest:
	  guest_name: str
	  guest_email: str
	  check_in: str
	  check_out: str
	  room_type: str
	  num_guests: int
	
	def create_booking(request: BookingRequest):
	  pass
	
	create_booking(BookingRequest("John", "john@mail.com", "2026-08-01", "2026-08-05", "Deluxe", 2))

Data Clumps

Smell: Different parts of the code contain identical groups of variables (such as parameters for connecting to a database) that should be turned into their own class.

Typical Fix: Fixed with Extract Class and Introduce Parameter Object.

Code with Smell	Refactored Code

def connect(host, port, username, password):
  pass
 	
def reconnect(host, port, username, password):
  pass
	class DBConfig:
	  def __init__(self, host, port, username, password):
	  self.host = host
	  self.port = port
	  self.username = username
	  self.password = password
	
	def connect(config: DBConfig):
	  pass
	
	def reconnect(config: DBConfig):
	  pass

Object-Orientation Abusers

All these smells are incomplete or incorrect application of object-oriented programming principles.

Switch Statements

Smell: You have a complex switch operator or sequence of if statements that select behavior based on a type code.

Typical Fix: Fixed with Replace Conditional with Polymorphism or Replace Type Code with Subclasses.

Code with Smell

```
def get_speed(bird_type):
    if bird_type == "European":
        return 35
    elif bird_type == "African":
        return 40
    elif bird_type == "Norwegian":
        return 30
```

Refactored Code

```
class Bird:
    def speed(self):
        raise NotImplementedError

class EuropeanSwallow(Bird):
    def speed(self):
        return 35

class AfricanSwallow(Bird):
    def speed(self):
        return 40

class NorwegianBlueParrot(Bird):
    def speed(self):
        return 30
```

Temporary Field

Smell: Temporary fields get their values, and thus are needed by objects, only under certain circumstances. Outside of these circumstances, they're empty.

Typical Fix: Fixed with Extract Class to move the field and its related methods to a smaller, dedicated class.

Code with Smell

```
class Calculation:
    def __init__(self):
        self.result = None  # only used during compute()

    def compute(self, a, b):
        self.result = a + b
        return self.result
```

Refactored Code

```
class AdditionCalculation:
    def compute(self, a, b):
        return a + b  # no temporary state stored on the object
```

Refused Bequest

Smell: If a subclass uses only some of the methods and properties inherited from its parent, the hierarchy is off-kilter. The unneeded methods may go unused or be redefined to give off exceptions.

Typical Fix: Fixed with Push Down Method, Push Down Field, or Replace Inheritance with Delegation.

Code with Smell	Refactored Code

`class Bird:`
`  def fly(self):`
`  print("Flying")`
 	
`class Penguin(Bird):`
`  def fly(self):`
`  raise NotImplementedError("Penguins can't fly")`
	`class Bird:`
	`  pass`
	
	`class FlyingBird(Bird):`
	`  def fly(self):`
	`  print("Flying")`
	
	`class Penguin(Bird):`
	`  pass  # no misleading fly() method inherited`

Alternative Classes with Different Interfaces

Smell: Two classes perform identical functions but have different method names, preventing them from being used interchangeably.

Typical Fix: Fixed with Rename Method and Move Method to unify their interfaces.

Code with Smell	Refactored Code

`class EmailSender:`
`  def send_message(self, msg):`
`  print(f"Email: {msg}")`
 	
`class SMSSender:`
`  def dispatch(self, msg):`
`  print(f"SMS: {msg}")`
	`class EmailSender:`
	`  def send(self, msg):`
	`  print(f"Email: {msg}")`
	
	`class SMSSender:`
	`  def send(self, msg):`
	`  print(f"SMS: {msg}")`

Change Preventers

These smells mean that if you need to change something in one place in your code, you have to make many changes in other places too. Program development becomes much more complicated and expensive as a result.

Divergent Change

Smell: You find yourself having to change many unrelated methods when you make changes to a class. For example, adding a new product type requires changing the methods for finding, displaying, and ordering products.

Typical Fix: Fixed with Extract Class to split the class along its separate responsibilities.

Code with Smell	Refactored Code

`class Product:`
`  def find(self): pass`
`  def display(self): pass`
`  def order(self): pass`
`  def calculate_tax(self): pass  # unrelated concern mixed in`
	`class Product:`
	`  def find(self): pass`
	`  def display(self): pass`
	`  def order(self): pass`
	
	`class TaxCalculator:`
	`  def calculate_tax(self, product): pass`

Shotgun Surgery

Smell: Making any modifications requires that you make many small changes to many different classes.

Typical Fix: Fixed with Move Method and Move Field to consolidate related changes into a single class.

Code with Smell	Refactored Code

`class OrderValidator:`
`  def check_amount(self, order): pass`
 	
`class OrderLogger:`
`  def log_amount_rule(self): pass`
 	
`class OrderNotifier:`
`  def notify_amount_rule_change(self): pass`
`  # one rule change forces edits across three classes`
	`class AmountRule:`
	`  def check(self, order): pass`
	`  def log(self): pass`
	`  def notify_change(self): pass`
	`  # rule logic centralized in one place`

Parallel Inheritance Hierarchies

Smell: Whenever you create a subclass for one class, you find yourself needing to create a subclass for another class as well.

Typical Fix: Fixed by making instances of one hierarchy refer to instances of another, then removing the hierarchy in one of them (often via Move Method/Move Field).

Code with Smell	Refactored Code

`class Engine: pass`
`class DieselEngine(Engine): pass`
`class PetrolEngine(Engine): pass`
 	
`class Car: pass`
`class DieselCar(Car): pass  # mirrors Engine hierarchy`
`class PetrolCar(Car): pass`
	`class Engine: pass`
	`class DieselEngine(Engine): pass`
	`class PetrolEngine(Engine): pass`
	
	`class Car:`
	`  def __init__(self, engine: Engine):`
	`  self.engine = engine  # composition instead of parallel subclassing`

Dispensables

A dispensable is something pointless and unneeded whose absence would make the code cleaner, more efficient and easier to understand.

Comments

Smell: A method is filled with explanatory comments, often compensating for unclear code rather than clarifying good code.

Typical Fix: Fixed with Extract Method and Rename Method so the code explains itself.

Code with Smell	Refactored Code

`def calc(a, b):`
`  # check if eligible for discount`
`  if a > 100:`
`  # apply 10 percent discount`
`  return b * 0.9`
`  return b`
	`def is_eligible_for_discount(amount):`
	`  return amount > 100`
	
	`def apply_discount(price):`
	`  return price * 0.9`
	
	`def calc(amount, price):`
	`  return apply_discount(price) if is_eligible_for_discount(amount) else price`

Duplicate Code

Smell: Two code fragments look almost identical, spread across the codebase.

Typical Fix: Fixed with Extract Method, and Pull Up Method for duplication across subclasses.

Code with Smell	Refactored Code

def print_invoice(order):
  total = sum(i.price * i.qty for i in order.items)
  print(f"Total: {total}")
 	
def print_receipt(order):
  total = sum(i.price * i.qty for i in order.items)
  print(f"Total: {total}")
	def calculate_total(order):
	  return sum(i.price * i.qty for i in order.items)
	
	def print_invoice(order):
	  print(f"Total: {calculate_total(order)}")
	
	def print_receipt(order):
	  print(f"Total: {calculate_total(order)}")

Lazy Class

Smell: Understanding and maintaining classes always costs time and money, so a class that doesn't do enough to earn your attention should be deleted.

Typical Fix: Fixed with Inline Class or Collapse Hierarchy.

Code with Smell	Refactored Code

class PhoneNumber:
  def __init__(self, number):
  self.number = number
 	
class Person:
  def __init__(self, name, number):
  self.name = name
  self.phone = PhoneNumber(number)
	class Person:
	  def __init__(self, name, number):
	  self.name = name
	  self.number = number

Data Class

Smell: A data class contains only fields and crude accessor methods (getters/setters). It is simply a container for data used by other classes and holds no independent behavior.

Typical Fix: Fixed with Move Method to relocate behavior into the data class, and Encapsulate Field/Collection.

Code with Smell	Refactored Code

class Order:
  def __init__(self, items, price):
  self.items = items
  self.price = price
	class Order:
	  def __init__(self, items, price):
	  self.items = items
	  self.price = price

`def get_total(order):` `  return order.price * len(order.items)`	`  def get_total(self):` `  return self.price * len(self.items)`

Dead Code

Smell: A variable, parameter, field, method, or class is no longer used, usually because it has become obsolete over time.

Typical Fix: Fixed by simply deleting the unused code.

Code with Smell	Refactored Code

`def calculate_total(price, tax, legacy_discount=None):`
`  # legacy_discount is never used anymore`
`  return price + tax`
	`def calculate_total(price, tax):`
	`  return price + tax`

Speculative Generality

Smell: There's an unused class, method, field, or parameter, created 'just in case' it's needed someday, that adds unnecessary complexity.

Typical Fix: Fixed with Collapse Hierarchy, Inline Class, Remove Parameter, or Rename Method.

Code with Smell	Refactored Code

`class ReportGenerator:`
`  def generate(self, data, future_format=None):`
`  # future_format never actually used by any caller`
`  return str(data)`
	`class ReportGenerator:`
	`  def generate(self, data):`
	`  return str(data)`

Couplers

All the smells in this group contribute to excessive coupling between classes, or show what happens when coupling is replaced by excessive delegation.

Feature Envy

Smell: A method accesses the data of another object more than its own data.

Typical Fix: Fixed with Move Method to relocate the method to the class whose data it uses most.

Code with Smell	Refactored Code

```
class Invoice:
    def print_customer_info(self, customer):
        print(customer.name)
        print(customer.address)
        print(customer.phone)
```

```
class Customer:
    def print_info(self):
        print(self.name)
        print(self.address)
        print(self.phone)
```

Inappropriate Intimacy

Smell: One class uses the internal fields and methods of another class, creating tight, unwanted coupling.

Typical Fix: Fixed with Move Method, Move Field, or Change Bidirectional Association to Unidirectional.

Code with Smell

```
class Engine:
    def __init__(self):
        self._rpm = 0

class Car:
    def accelerate(self, engine: Engine):
        engine._rpm += 100  # reaching into Engine internals
```

Refactored Code

```
class Engine:
    def __init__(self):
        self._rpm = 0

    def increase_rpm(self, amount):
        self._rpm += amount

class Car:
    def accelerate(self, engine: Engine):
        engine.increase_rpm(100)
```

Message Chains

Smell: In code you see a series of calls resembling a->b()->c()->d(), tightly coupling the client to the structure of intermediate objects.

Typical Fix: Fixed with Hide Delegate.

Code with Smell

```
manager_name = person.get_department().get_manager().get_name()
```

Refactored Code

```
class Person:
    def manager_name(self):
        return self.department.manager.name

manager_name = person.manager_name()
```

Middle Man

Smell: If a class performs only one action, delegating all its work to another class, why does it exist at all?

Typical Fix: Fixed with Remove Middle Man.

Code with Smell	Refactored Code

class PersonProxy:
  def __init__(self, person):
  self._person = person
 	
  def get_name(self):
  return self._person.get_name()
 	
name = proxy.get_name()
	name = person.get_name()